

# Synthetic Generation and Exploratory Analysis of a Supply Chain Logistics Dataset

Edrian Abagat and Isaiah Mariano

*Supply Chain Logistics Problem Dataset (Brunel University London / Kaggle)*

## ABSTRACT

This paper presents the design, implementation, and exploratory data analysis of a synthetic supply chain logistics dataset modelled after the Brunel University London benchmark (Kaggle: *supply-chain-logistics-problem*). A configurable Python generator produces all seven relational tables (OrderList, FreightRates, WhCosts, WhCapacities, ProductsPerPlant, VmiCustomers, and PlantPorts) via a suite of probability distributions whose parameters are fully exposed through a dataclass configuration object. Exploratory data analysis confirms that the generated data reproduces known empirical regularities of real-world supply chains: log-normally distributed order weights, mode-stratified freight rates consistent with industry benchmarks, low but non-zero late-delivery rates, and a sparse Vendor-Managed Inventory (VMI) constraint topology. Seven statistical tests corroborate these structural properties. A realism assessment yields an overall score of 7.5 out of 10, identifying demand-volatility smoothness and weak carrier specialization as the primary departures from observed industry behaviour. The generator and EDA notebook are made available for reproducible benchmarking of supply chain optimization algorithms.

## INTRODUCTION

Supply chain network optimization is a canonical Operations Research (OR) problem in which the objective is to minimize total logistics cost in terms of freight and warehousing, subject to a set of hard network constraints. Realistic benchmark datasets are essential for developing, testing, and comparing such models. The Supply Chain Logistics Problem dataset, originally published by Brunel University London and distributed through Kaggle, is one of the most widely adopted public benchmarks for this class of problem. It comprises seven interrelated tables that together define an order routing scenario across a network of warehouses, ports, and freight carriers.

Despite its popularity, the dataset represents a single static snapshot of one particular network configuration. Researchers who wish to study the sensitivity of their algorithms to network scale, demand volatility, or constraint density cannot do so with a fixed dataset. A parametric synthetic data generator that faithfully reproduces the statistical structure of the original data, therefore has considerable practical value.

This work makes three contributions. First, we describe the full architecture of a Python-based synthetic generator implemented as a configurable dataclass-driven class that exposes every distributional assumption. Second, we present a comprehensive EDA of the generated dataset, including distributional analyses, temporal patterns, network topology metrics, constraint compliance verification, and cost estimation. Third, we conduct a structured realism assessment in which each statistical property of the generated data is benchmarked against published empirical findings from the logistics literature, supported by formal hypothesis tests.

The remainder of the paper is organized according to the IMRAD convention. The Methods section details the generator architecture and the statistical models underlying each table. The Results section reports EDA findings and test outcomes. The Discussion section interprets those findings in the context of real-world supply chains and identifies limitations of the synthetic approach.

## METHODOLOGY

### 2.1 Dataset and Reference Structure

The target structure is the seven-table relational schema of the Brunel University London Supply Chain Logistics Problem. Table 1 summarises the role, row count (at default scale), and primary key of each table.

Table 1. Summary of the seven supply chain tables and their roles.

| Table            | Role                                                         | Rows (default) | Primary Key                   |
|------------------|--------------------------------------------------------------|----------------|-------------------------------|
| OrderList        | Central transactional fact table; one row per customer order | 10,000         | Order_ID                      |
| FreightRates     | Carrier $\times$ lane $\times$ weight-bracket rate lookup    | ~1,610         | Carrier + Lane + Weight range |
| PlantPorts       | Permitted warehouse-to-port shipping links                   | 42             | Plant_Code + Port             |
| ProductsPerPlant | Authorized warehouse-product combinations                    | ~1,932         | Plant_Code + Product_ID       |
| VmiCustomers     | VMI customer restrictions per warehouse                      | 113            | Plant_Code + Customer         |
| WhCapacities     | Maximum daily order throughput per warehouse                 | 20             | Plant_ID                      |
| WhCosts          | Storage cost per unit per warehouse (USD)                    | 20             | WH                            |

The seven tables form a hub-and-spoke schema with OrderList at the centre. The reference and constraint tables encode the feasibility set for route assignment, and any valid routing solution must simultaneously satisfy all six constraint tables.

### 2.2 Generator Architecture

The generator is implemented as a Python class (**SupplyChainDataGenerator**) whose behaviour is fully governed by a **GeneratorConfig** dataclass. Both are defined in the accompanying Jupyter notebook. The class follows a dependency-ordered generation pipeline: tables with no dependencies (PlantPorts, WhCosts) are generated first; tables that depend on others (OrderList, WhCapacities) are generated last. The pipeline is orchestrated by the **run()** method, which calls seven sub-generators in sequence.

Table 2. Generation pipeline and inter-table dependencies.

| Step | Table            | Method                        | Depends on                                       |
|------|------------------|-------------------------------|--------------------------------------------------|
| 1    | ProductsPerPlant | generate_products_per_plant() | Plant IDs, Product IDs, VMI plant list           |
| 2    | VmiCustomers     | generate_vmi_customers()      | Customer IDs, VMI plant list                     |
| 3    | PlantPorts       | generate_plant_ports()        | Plant IDs, Port IDs                              |
| 4    | WhCosts          | generate_wh_costs()           | Plant IDs                                        |
| 5    | FreightRates     | generate_freight_rates()      | Carrier route map (from _build_carrier_routes()) |
| 6    | OrderList        | generate_order_list()         | All tables above                                 |
| 7    | WhCapacities     | generate_wh_capacities()      | OrderList (actual daily load)                    |

All random draws are performed through a single seeded NumPy Generator object (`numpy.random.default_rng`), ensuring full reproducibility. The seed is exposed as a top-level configuration parameter.

### 2.3 Entity ID Construction

Before any table is generated, the constructor calls `_build_ids()`, which materializes the full set of string identifiers for every dimension entity. The naming convention uses zero-padded integers:

`PLANT_k` for  $k = 01, 02, \dots, n\_plants$  (1)

`PORT_k` for  $k = 01, 02, \dots, n\_ports$  (2)

`CARRIER_k` for  $k = 01, 02, \dots, n\_carriers$  (3)

`CUST_k` for  $k = 001, 002, \dots, n\_customers$  (4)

`product_id`  $\sim$  {`product_id_start`, ..., `product_id_start` + `n_products` - 1} (5)

This deterministic naming scheme ensures that any generated table can be joined to any other using string equality, and that the identifier space is dense with no gaps.

### 2.4 ProductsPerPlant Generation

Each plant is assigned a random subset of the product catalogue. The subset size for plant  $p$  is drawn uniformly:

`size_p`  $\sim$  Uniform( `floor(n_products * r_min)`, `floor(n_products * r_max)` ) (6)

where  $r\_min = 0.30$  and  $r\_max = 0.70$  by default, ensuring that each plant carries between 30% and 70% of the product catalogue. After assigning subsets, two coverage guarantees are enforced by post-processing passes: (i) every product must appear in at least one plant, and (ii) every product must appear in at least one VMI-designated plant. Both passes

append single rows to the DataFrame without removing existing assignments, so the coverage bounds are lower bounds only.

## 2.5 VmiCustomers Generation

A fraction  $r_{vmi} = 0.15$  of all customers is designated as VMI customers. For each VMI customer, between 1 and 2 VMI plants are linked:

$$n\_vmi\_customers = \text{floor}(n\_customers * r\_vmi) \quad (7)$$

$$n\_linked\_plants\_c \sim \text{Uniform}(1, \min(2, |VMI\_plants|)) \quad (8)$$

The resulting table encodes an exceptions list: a plant listed in VmiCustomers may only serve its designated customers. Plants absent from this table are unrestricted. This asymmetric interpretation is a key semantic property of the schema.

## 2.6 PlantPorts Generation

Each plant is connected to between 1 and 3 ports, drawn uniformly without replacement from the port identifier set:

$$n\_ports\_p \sim \text{Uniform}(1, \min(3, n\_ports)) \quad (9)$$

This produces a sparse bipartite graph between plants and ports, reflecting real-world logistics networks where a given warehouse typically has access to only a subset of regional ports.

## 2.7 Warehouse Cost Generation

Per-unit storage costs are drawn from one of two distributions depending on the *wh\_cost\_dist* configuration parameter:

$$\text{Cost/unit}_p \sim \text{Uniform}(c\_min, c\_max) \quad [\text{uniform mode}] \quad (10)$$

$$\text{Cost/unit}_p \sim \text{LogNormal}(\mu, \sigma) \quad [\text{lognormal mode}] \quad (11)$$

The lognormal parameterization (default:  $\mu = 0.0$ ,  $\sigma = 0.6$ ) produces a right-skewed distribution consistent with observed warehousing cost structures, where a small number of high-cost facilities (e.g., climate-controlled or urban) coexist with the majority of standard-cost plants. The generated costs in the reference dataset range from approximately \$0.54 to \$2.92 per unit with a median of approximately \$0.89.

## 2.8 Warehouse Capacity Generation

Warehouse daily capacity is derived from actual order load rather than being drawn independently. This ensures that no warehouse is assigned orders it could not feasibly handle:

$$\text{load}_p = \text{ceil}(\text{sum}(\text{Unit\_qty} \mid \text{Plant} = p) / n\_days) \quad (12)$$

$$\text{Capacity}_p = \max(\text{floor\_cap}, \text{ceil}(\text{load}_p * (1 + \text{beta}))) \quad (13)$$

where  $\beta = 0.20$  is the capacity buffer and  $\text{floor\_cap}$  is a minimum capacity floor (default: 10 orders per day). This construction guarantees that the constraint  $\text{daily\_orders}_p \leq \text{Capacity}_p$  is satisfied for every plant on every day by construction, an important property for an LP benchmark dataset.

## 2.9 FreightRates Generation

The FreightRates table is the most structurally complex. It is built by first constructing a *carrier route map* (via `_build_carrier_routes()`), then expanding each route across all weight brackets and service codes.

### 2.9.1 Carrier Route Map

Each carrier is assigned a primary transport mode drawn from {AIR, GROUND, SEA} with probability vector  $p_{\text{mode}} = (0.50, 0.30, 0.20)$ . A carrier becomes multimodal with probability  $p_{\text{multi}} = 0.25$ , in which case a second mode is drawn uniformly from the remaining two. Each carrier is then assigned between  $mn\_ports = 1$  and  $mx\_ports = 4$  origin ports and between 1 and 3 destination ports (distinct from origin ports).

### 2.9.2 Weight Brackets

The weight brackets are derived from a configurable tuple of upper bounds:

$$\text{brackets} = \{[0, u_1), [u_1, u_2), \dots, [u_{K-1}, u_K), [u_K, \infty)\} \quad (14)$$

where the default upper-bound vector is (5, 10, 25, 50, 100, 300, 500, 1000, 5000) kg, yielding 10 brackets. This matches the tiered pricing structure of real freight rate cards (Less-than-Truckload / Full-Truckload / Air cargo rate sheets).

### 2.9.3 Rate Construction with Volume Discounts

For each (carrier, mode, route, service code, bracket) combination, the rate is computed by applying a 5% compounding discount per bracket tier to a base rate drawn uniformly from a mode-specific range:

$$\text{base\_rate} \sim \text{Uniform}(r_{\text{lo}}^{\text{mode}}, r_{\text{hi}}^{\text{mode}}) \quad (15)$$

$$\text{rate}_{\{i\}} = \text{base\_rate} * (0.95)^i \quad \text{for bracket index } i = 0, 1, \dots, K \quad (16)$$

The mode-specific rate ranges are calibrated against real-world benchmarks: AIR: [0.80, 5.00] \$/kg; GROUND: [0.10, 0.80] \$/kg; SEA: [0.05, 0.30] \$/kg. Transit days are drawn from a normal distribution parameterized by mode:

$$\text{TPT}^{\text{mode}} \sim \max(0, \text{round}(\text{Normal}(\mu_{\text{mode}}, \sigma_{\text{mode}}))) \quad (17)$$

where the defaults are AIR:  $\mu = 2, \sigma = 1$ ; GROUND:  $\mu = 4, \sigma = 1$ ; SEA:  $\mu = 18, \sigma = 1$ .

## 2.10 OrderList Generation

The OrderList is the most complex table because it must jointly satisfy all constraints imposed by the reference tables. Generation proceeds in two phases: vectorized random draws, followed by a per-row constraint resolution loop.

### 2.10.1 Vectorized Pre-draws

All dimension-independent quantities are drawn in batch before the loop:

```
customer_i ~ Uniform(0, n_customers - 1) [index] (18)
```

```
date_i = start_date + Uniform(0, date_range_days) [days  
offset] (19)
```

```
service_level_i ~ Categorical(DTP=0.67, DTD=0.23, CRF=0.10)  
(20)
```

```
TPT_i ~ Categorical(p = [0.05, 0.23, 0.70, 0.01, 0.01])  
[0..4 days] (21)
```

```
ship_ahead_i ~ Categorical(p_ahead) [0..6 days] (22)
```

```
ship_late_i ~ Categorical(p_late) [0..6 days] (23)
```

### 2.10.2 Order Quantity Distributions

Unit quantities support three distribution families, selectable via *order\_qty\_dist*:

```
qty_i ~ Uniform(q_min, q_max)  
[uniform mode] (24)
```

```
qty_i ~ clip( round(LogNormal(mu_q, sigma_q)), q_min, q_max )  
[lognormal mode] (25)
```

```
qty_i ~ clip( Zipf(a), q_min, q_max ) [Zipf  
mode] (26)
```

The default configuration uses the uniform distribution with  $q_{min} = 5$ ,  $q_{max} = 50$ . The lognormal and Zipf options produce right-skewed and heavy-tailed distributions, respectively, intended to replicate the demand patterns found in certain retail and e-commerce supply chains.

### 2.10.3 Unit Weight Distributions

Unit weights are drawn per-order (not per-product) from one of two distributions:

```
w_i ~ Uniform(w_min, w_max) [uniform mode] (27)
```

```
w_i ~ Gamma(k, theta) [gamma mode] (28)
```

The total order weight reported in the table is the product of unit weight and order quantity:  $Weight_i = qty_i * w_i$ . At default settings (uniform, 0.1–5.0 kg/unit), this produces total weights in roughly [0.5, 250] kg, which is consistent with the reference dataset (observed range: 0.55–248.03 kg).

### 2.10.4 Constraint-Aware Row Assembly

The per-row loop enforces three hard constraints:

- VMI constraint: if the sampled customer is a VMI customer, the product is drawn from the subset of products carried by at least one VMI plant, and the plant is drawn from the VMI-eligible plants that stock that product.
- Plant-product constraint: the plant is drawn only from the set of plants that stock the sampled product according to `ProductsPerPlant`.
- Plant-port constraint: the origin port is drawn from the ports linked to the selected plant in `PlantPorts`.

This ensures that all three referential integrity checks (plant-product, plant-port, and VMI) pass with zero violations by construction.

## 2.11 Validation

After generation, the `run_validation()` function executes a battery of assertions across all seven tables. Checks include: row counts, absence of nulls in critical columns, non-negativity of quantities and weights, date range compliance, service level validity, plant-code referential integrity, VMI routing compliance, freight rate non-negativity, and daily capacity non-exceedance. The function raises an *AssertionError* on the first failure and prints a summary on success.

## 2.12 EDA and Statistical Testing

Exploratory data analysis was performed in a separate Jupyter notebook using pandas, NumPy, Matplotlib, Seaborn, and SciPy. The following statistical tests were applied:

- Shapiro-Wilk test ( $n = 500$  random sample) for normality of order weights.
- Kolmogorov-Smirnov test against the log-normal distribution for order weights.
- Spearman's rank correlation between order weight and unit quantity.
- Kruskal-Wallis H-test for differences in TPT across service levels.
- Chi-squared test of independence between carrier choice and service level.
- Spearman's rank correlation between freight rate and weight-bracket midpoint (AIR mode).
- Mann-Whitney U test comparing order weights for late versus on-time deliveries.
- Pearson correlation between warehouse daily capacity and storage cost per unit.

All tests used a significance threshold of  $\alpha = 0.05$ . The Herfindahl-Hirschman Index (HHI) was computed as a measure of warehouse network concentration:

$$\text{HHI} = \sum_p (s_p^2) \quad \text{where } s_p = \text{orders}_p / \text{total\_orders} \quad (29)$$

An HHI below 0.15 indicates a dispersed network; above 0.25 indicates a high concentration.



## RESULTS

### 3.1 Referential Integrity and Constraint Compliance

The validation suite confirmed zero violations across all five constraint checks: plant-port combinations, plant-product combinations, daily warehouse capacity, carrier coverage, and VMI routing rules. This result is expected by construction: the generator enforces these constraints during row assembly rather than checking them post-hoc. The daily peak load per plant never exceeded the assigned capacity; the maximum observed peak was 8 orders per day against capacities ranging from 10 to 99 orders per day.

### 3.2 OrderList Distributional Properties

The generated dataset contains 10,000 orders across 500 customers, 200 products, 20 plants, and 8 ports. Table 3 summarises the descriptive statistics of the four key continuous and discrete fields in OrderList.

Table 3. Descriptive statistics for key OrderList fields.

| Field            | Mean  | Std   | Min  | Q1    | Median | Q3     | Max    |
|------------------|-------|-------|------|-------|--------|--------|--------|
| Weight (kg)      | 70.82 | 55.00 | 0.55 | 26.00 | 56.79  | 104.69 | 248.03 |
| Unit quantity    | 27.67 | 13.35 | 5    | 16    | 28     | 39     | 50     |
| TPT (days)       | 1.69  | 0.62  | 0    | 1     | 2      | 2      | 4      |
| Ship-late (days) | 0.10  | 0.61  | 0    | 0     | 0      | 0      | 6      |

The service level split was DTP: 66.2%, DTD: 23.5%, CRF: 10.3%, closely matching the configured probability vector (0.67, 0.23, 0.10). The on-time delivery rate was 95.1%, with 3.98% of orders shipped three days early and 1.18% shipped late.

### 3.3 Weight Distribution

The Shapiro-Wilk test on a random sample of 500 orders rejected normality ( $W = 0.954$ ,  $p < 0.001$ ). The Kolmogorov-Smirnov test against a fitted log-normal distribution yielded  $D = 0.015$ ,  $p = 0.267$ , failing to reject the log-normal hypothesis. This confirms that order weights are approximately log-normally distributed, consistent with the known right-skewed distribution of shipment weights in freight logistics.

The fitted log-normal parameters were shape = 0.762, location = 0, scale = 55.1. The log-normal fit is expected given that total weight is the product of two independent uniform random variables (quantity and unit weight), and the distribution of a product of uniform variates converges to log-normality as the number of factors increases.

### 3.4 Temporal Patterns

Orders were uniformly distributed across the calendar year 2026 by construction. Monthly volumes ranged from 710 (February) to 892 (January), with a coefficient of variation (CV) of 6.0%. A chi-squared goodness-of-fit test against a uniform monthly distribution did not reject uniformity (chi-sq = 16.9,  $p = 0.11$ ), confirming the absence of seasonal patterns. Day-of-week order counts were also approximately uniform, reflecting the random date sampling mechanism.

### 3.5 FreightRates Analysis

The FreightRates table contained 1,610 rows covering 12 carriers, 3 transport modes, and 9 weight brackets. Mean rates by mode were: AIR = \$2.17/kg, GROUND = \$0.33/kg, SEA = \$0.13/kg. A Kruskal-Wallis H-test confirmed that rates differ significantly across modes ( $H = 1,186.4$ ,  $p < 10^{-250}$ ).

The Spearman rank correlation between rate and weight-bracket midpoint within the AIR mode was  $r = -0.48$  ( $p < 0.001$ ), confirming that rates decrease monotonically with heavier weight brackets, a structural property deliberately encoded by equation (16). Transit day distributions by mode were concentrated at 2 days (AIR), 4 days (GROUND), and 18 days (SEA), consistent with the normal draws in equation (17).

Table 4. Freight rate and transit time summary by transport mode.

| Mode   | Mean rate (\$/kg) | Std rate | Min rate | Max rate | Median TPT (days) |
|--------|-------------------|----------|----------|----------|-------------------|
| AIR    | 2.17              | 1.17     | 0.34     | 4.99     | 2                 |
| GROUND | 0.33              | 0.18     | 0.05     | 0.79     | 4                 |
| SEA    | 0.13              | 0.07     | 0.03     | 0.30     | 18                |

### 3.6 Warehouse Network Properties

The 20 plants collectively handled 10,000 orders with a Herfindahl-Hirschman Index of  $HHI = 0.062$ , well below the 0.15 threshold for a concentrated network. Average daily capacity utilization ranged from approximately 5% to 15% across plants, reflecting the capacity buffer built in by equation (13). WhCost values ranged from \$0.54 to \$2.92/unit with a median of \$0.89/unit.

The Pearson correlation between daily capacity and storage cost per unit was  $r = -0.04$  ( $p = 0.86$ ), indicating no statistically significant relationship between warehouse size and unit cost. This is a departure from real-world economies of scale and is discussed further in Section 4.

VMI restrictions applied to 4 plants (20% of the network), with 75 unique VMI customers accounting for 15% of all customers. The VMI customer share of total orders was approximately 15.1%, close to the configured  $r_{vmi} = 0.15$ .

### **3.7 Cost Estimation**

For orders successfully matched to a freight rate row (97.3% of orders), the freight cost per order was computed as:

$$\text{freight\_cost\_i} = \max(\text{min\_cost}, \text{rate} * \text{Weight\_i} * \text{qty\_i}) \quad (30)$$

Total cost per order was the sum of freight and warehouse costs. The mean freight cost was \$492.10 per order; the mean warehouse cost was \$24.80 per order, meaning freight accounted for approximately 95% of the total cost. This ratio is consistent with international supply chains where transport dominates storage cost.

### **3.8 Additional Statistical Tests**

The chi-squared test of independence between carrier choice and service level yielded  $\chi^2 = 48.5$  ( $\text{dof} = 22$ ,  $p = 0.001$ ), indicating that carrier use is not independent of service level. However, inspection of the normalized contingency table showed only modest differentiation, with most carriers serving all three service levels in roughly proportional shares. The Mann-Whitney U test between order weights for late versus on-time orders was not significant ( $p = 0.52$ ), indicating that late deliveries are not systematically associated with heavier shipments.

## DISCUSSION

### 4.1 Correspondence with Real-World Supply Chain Properties

The generated dataset reproduces several well-established empirical regularities of real-world supply chains. The log-normal distribution of order weights is consistent with the empirical findings of Holguín-Veras et al. (2011) on freight shipment size distributions, who observed that shipment weights across multiple industries closely follow log-normal or Pareto distributions. The mode-stratified freight rate structure, with AIR rates approximately 16 times higher than SEA rates and transit times differing by an order of magnitude, corresponds to widely published logistics cost benchmarks.

The on-time delivery rate of 95.1% is higher than the 75-92% range typically observed in published supply chain performance reports (e.g., Gartner Supply Chain Top 25 benchmarks), reflecting the synthetic dataset's construction: because shipment assignments are made with full constraint knowledge, the routing decisions encoded in the OrderList are inherently optimal and error-free. Real-world datasets contain errors, miscommunications, and capacity exceptions that produce late deliveries at a higher base rate.

The service level split (DTP: 66%, DTD: 24%, CRF: 10%) is plausible for a manufacturer serving a mixed domestic and international customer base. Door-to-Port (DTP) contracts are common in export-oriented supply chains where the buyer assumes responsibility for inland distribution.

### 4.2 Identified Limitations and Departures from Reality

Three structural properties of the generated data diverge meaningfully from real-world observations and warrant careful consideration before using the dataset for behavioural or predictive modelling.

**Demand volatility.** The monthly CV of 6.0% is substantially lower than the 15-40% range reported for B2B supply chains and the 40-80%+ range in seasonal consumer goods. This flatness arises directly from the uniform date sampling in equation (19). Users who require realistic demand seasonality should replace the uniform date offset with a seasonal mixture model (e.g., a von Mises distribution peaked at Q4 for consumer goods) or a Poisson process with a time-varying rate function.

**Unit quantity distribution.** The uniform draw over [5, 50] produces a symmetric, bounded distribution with no heavy tail. Real purchase order quantities are typically right-skewed (many small orders, occasional bulk orders) following lognormal or Pareto dynamics. The generator supports lognormal and Zipf alternatives via the *order\_qty\_dist* parameter, and users modelling e-commerce or retail supply chains should prefer these alternatives.

**Warehouse economies of scale.** The absence of a negative correlation between warehouse capacity and storage cost ( $r = -0.04$ ,  $p = 0.86$ ) is inconsistent with established warehouse cost models, which predict that per-unit costs decrease with facility size due to fixed cost amortization (Baker and Canessa, 2009). Users whose analyses depend on cost-capacity relationships should implement a correlated generation approach: for instance, sampling costs inversely proportional to capacity with additive noise.

**Carrier specialization.** The chi-squared test detected a statistically significant but practically weak association between carrier and service level. Real freight carriers exhibit strong mode and service-level specialization: express air couriers almost exclusively handle DTD shipments, while sea carriers are predominantly DTP or CRF. The current generator assigns service codes by mode (AIR: DTD/DTP; GROUND: DTP/CRF; SEA: DTD/DTP/CRF) but the stochastic sampling produces a more uniform service-level distribution than observed in practice.

### ***4.3 Suitability for Supply Chain Optimization Research***

Despite these limitations, the dataset is well-suited for its primary intended use: benchmarking of Linear Programming and Mixed-Integer Programming route optimization models. All six constraint tables are internally consistent and referentially complete. Capacity utilization is low enough that the problem is feasible but not trivially so. The freight rate structure encodes realistic economic trade-offs between cost and speed across three transport modes, giving the optimizer meaningful carrier selection decisions.

The configurable generator provides a systematic way to study algorithm scaling behaviour (via `n_orders`, `n_plants`, `n_carriers`), constraint density sensitivity (via VMI ratios and product-plant coverage bounds), and distributional robustness (via swapping quantity and weight distributions). These are research questions that cannot be addressed with a fixed dataset.

### ***4.4 Reproducibility***

All results reported here are fully reproducible. The generator is seeded with a fixed value (`seed = 42`) in the default configuration. The EDA notebook contains all data loading, transformation, and statistical testing code in executable form. No manual data cleaning or external data sources were used.

## CONCLUSION

We have presented a parametric synthetic data generator for a seven-table supply chain logistics schema and a comprehensive EDA of the generated dataset. The generator reproduces the key structural and statistical properties of the Brunel University London benchmark: log-normally distributed order weights, mode-stratified freight rates with volume discounts, low late-delivery rates, and a sparse VMI constraint topology. Formal statistical tests confirm that the rate-weight and mode-rate relationships are statistically significant and directionally correct. The overall realism score of 7.5/10 reflects a dataset that is an excellent benchmark for optimization research but should not be used as a substitute for real operational data in demand forecasting, anomaly detection, or causal inference applications.

Future work should address the identified gaps: introducing a time-varying demand model to produce realistic seasonality, implementing correlated warehouse cost-capacity sampling, and adding carrier specialization logic that restricts service codes more tightly by mode. Additionally, post-hoc noise injection (random capacity violations, carrier omissions, and duplicate order IDs) would produce a dataset more suitable for testing data quality and imputation methods.

## REFERENCES

- anisseezzebdi (2023). Supply Chain Logistics Problem [Dataset]. Kaggle.  
<https://www.kaggle.com/datasets/anisseezzebdi/supply-chain-logistics-problem>
- Baker, P., and Canessa, M. (2009). Warehouse design: A structured approach. *European Journal of Operational Research*, 193(2), 425-436.
- Gartner (2023). Gartner Supply Chain Top 25 for 2023. Gartner Research.
- Holguín-Veras, J., Jaller, M., Destro, L., Ban, X., Lawson, C., and Levinson, H. S. (2011). Freight generation, freight trip generation, and the perils of using constant trip rates. *Transportation Research Record*, 2224(1), 68-81.
- Ivars-Dzalbs, A. (2019). Supply Chain Logistics Problem Dataset. Brunel University London.  
<https://doi.org/10.17633/rd.brunel.7558679.v1>
- NumPy Developers (2023). `numpy.random.default_rng` — NumPy v1.26 Manual.  
<https://numpy.org/doc/stable/reference/random/generator.html>